

Project Intro

Accelerating Lossless Compression & Adaptive Arithmetic Coding

AUTHORS

Rory Self, Bryan Bong,
Ujjwal Uberoi & Sadat Kabir

Problem Outline

01

Lossless Compression: Widely used in file storage, image/video codecs, networking, and embedded systems to reduce storage and transmission costs.

02

Arithmetic Coding: Achieves high compression efficiency by encoding symbols based on probability distribution, outperforming simpler entropy coders like Huffman.

03

Computational Bottleneck: Adaptive coding requires repeated interval updates and probability model maintenance for every input symbol, limiting software throughput.

04

Proposed FPGA Solution: Investigating whether hardware pipelining and streaming via High-Level Synthesis (HLS) can scale throughput while maintaining perfect compression fidelity.



Project Aims

- Develop a software reference implementation of adaptive arithmetic coding.
- Design and implement an HLS hardware accelerator targeting the Kria KV260 FPGA.
- Optimise the accelerator using FPGA-oriented techniques such as pipelining, streaming interfaces and custom-width arithmetic.
- Verify functional correctness against the software implementation.
- Benchmark throughput, latency and FPGA resource utilisation to evaluate the effectiveness of hardware acceleration.



Arithmetic Coding Algorithm

A lossless method that compresses an entire message into ONE number in the interval $[0, 1)$.

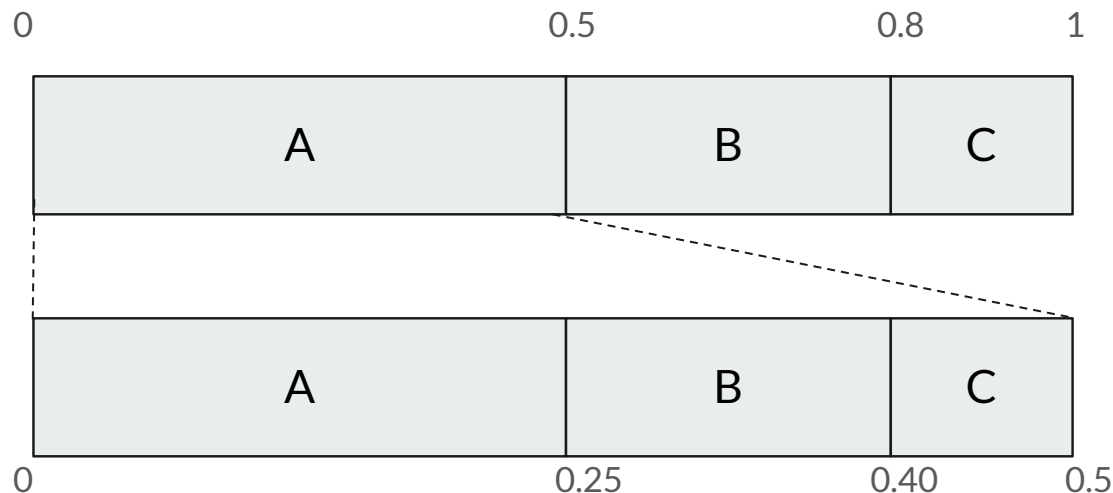
How does it work?

- **Probability Line**
 - The algorithm starts with an initial interval of $[0, 1)$. This range is divided into sub-Intervals proportional to the probability of each symbol in the alphabet.
- **Successive Sub-division**
 - As each character in the input message is read, the algorithm focuses entirely on that character's specific sub-interval.
 - This sub-interval becomes the new base range, which is again subdivided using the same symbol probabilities.
- **The Final Number**
 - As the message gets longer, the interval narrows and becomes smaller. The final output is a single value that falls anywhere inside this narrow range.
 - Highly probable strings narrow the interval more slowly, requiring fewer bits to represent.

Worked example (encoding “AB”)

$P(A) = 0.5$, $P(B) = 0.3$, $P(C) = 0.2$

Each bar is a zoomed in view of one slice above it



1. Read A.

Keep A's slice $\rightarrow [0-0.5)$

2. Read B.

Keep B's sub slice $\rightarrow [0.25, 0.40)$

3. Send any number inside $[0.25, 0.40) \rightarrow \text{"AB"}$

Investigation Findings



Based on initial research, implementing a form of arithmetic coding on the Kria should provide a fulfilling acceleration challenge given the following considerations:

- Lossless compression algorithms usually provide substantial opportunity for improvement via hardware acceleration. However, parallelising arithmetic coding loads provides unique challenges.
 - Interval subdivision in arithmetic coding is an inherently sequential operation and has limited scope for parallelism.
 - Instead, a focus can be made on pipelining and parallelism of algorithmic components, such as generation of the probability model.
- The probability model can be implemented either statically or dynamically.
 - Static: symbols (e.g. '0' or '1') have a set probability / sub-interval. Inefficient if real frequency differs.
 - Adaptive / Dynamic: symbol probability is generated based on the payload. This is the ideal approach.
- Open-source implementations of arithmetic coding have been collected and can be compared to an accelerated solution if profiled on the Kria's ARM processor. e.g. <https://github.com/nayuki/Reference-arithmetic-coding/blob/master/cpp/ArithmeticCompress.cpp>

Project Plan



Plan: Implementation of an accelerated, adaptive arithmetic coding pipeline on the Kria KV260.

Milestones & Timeline:

1. **Successful synthesis of a standard arithmetic encoding algorithm** derived from traditional C++ solutions.
Wednesday, July 15 (Week 7)
2. **Creation of a corresponding decoding algorithm** to test validity of encoder and lossless data integrity.
Wednesday July 22 (Week 8)
3. **Enhancement of the HLS kernel** by implementing parallel frequency model generation and pipelined interval subdivision.
Monday July 27 (Week 9)
4. **Performance benchmarking of the algorithms**, comparison of accelerated throughput to open-source solutions running on the ARM processor.
Friday July 31 (Week 9)

Updates will be maintained on the group GitHub repository: <https://github.com/rory-self/comp4601-project>



Risk Management

- **Sequential dependency in arithmetic coding limits achievable speedup:** Scope the accelerated kernel early, update probability model; before committing to an architecture we can validate with Amdahl's Law
- **Correctness regressions between SW/HW output:** Maintain a bit-exact SW reference throughout; can then automate comparison testing on every HW iteration
- **Model choice changes the entire kernel boundary:** Profile before finalizing the kernel boundary. Run the SW reference with a profiler to get cycle breakdowns between model and coder update.
- **Carry propagation adds unexpected hardware complexity:** Build a stress-test input set in software before HLS, can catch correctness bugs on the SW reference first, not after synthesis.



Team Member Roles

Rory Self (Algorithm & SW Profiling Lead): Implement the reference arithmetic coder in C/C++, identify hot loop, define the kernel boundary for acceleration

Sadat Kabir((HLS/PL Design) : Focuses on hardware kernel, design and implement the HLS kernel, manage pipelining, driving timing closure

Ujjwal Uberoi (Integration & Verification Lead): Mainly work on PS-PL interface, AXI-stream/memory-mapped interface design, driver code on the PS side, do simulation and on-board verification, also test correctness testing against SW reference

Bryan Bong (Performance/Power Analysis & Project Management): Enhance benchmark and energy (power measurement), help working on correctness verification as well, track milestones based on Week 5 plan with Week 10 deliverables, coordinate report and presentation assembly.